

AutoChip Tutorial Assignment

Jiangyue Chu jc13605

Part I (a)

Example1 binary_to_bcd:

In this example, I used AutoChip to generate a combinational module named binary_to_bcd_converter. The module converts a 5-bit binary input (0–31) into two-digit BCD format.

```
# 1) prompt
prompt_text = textwrap.dedent("""
I want to write a Verilog-2001 module named binary_to_bcd_converter.

It converts a 5-bit binary number (0..31) into two-digit BCD.

Interface must be exactly:

module binary_to_bcd_converter(
    input  [4:0] binary_input,
    output reg [7:0] bcd_output
);

Requirements:
- bcd_output[7:4] = tens
- bcd_output[3:0] = ones
- Combinational only (always @(*))
- Synthesizable
- No delays, no initial blocks
- Output ONLY the module code
""").strip()

... /content/binary_to_bcd_example
total 36K
drwxr-xr-x 3 root root 4.0K Feb 16 06:08 .
drwxr-xr-x 1 root root 4.0K Feb 16 01:53 ..
-rw-r--r-- 1 root root 658 Feb 16 06:11 binary_to_bcd.sv
-rw-r--r-- 1 root root 1.2K Feb 16 06:11 binary_to_bcd_tb.sv
-rw-r--r-- 1 root root 320 Feb 16 06:11 config.json
drwxr-xr-x 8 root root 4.0K Feb 16 01:54 outputs_bcd
-rw-r--r-- 1 root root 11K Feb 16 04:54 wave.vcd
Iteration: 0
Model type: ChatGPT
Model ID: gpt-4o-mini
Number of responses: 1
Testbench ran successfully
Mismatches: 0
Samples: 32
Cost for response 0: $0.0003847500
Response ranks: [1.0]
Response lengths: [1296]

Time to Generate: 18.63 seconds
Best response summary:
<__main__.LLMResponse object at 0x7daa70fc76e0>

Find generated modules:
outputs_bcd/iter0/response0/binary_to_bcd_converter.sv
outputs_bcd/iter1/response0/binary_to_bcd_converter.sv
outputs_bcd/iter2/response0/binary_to_bcd_converter.sv
outputs_bcd/iter3/response0/binary_to_bcd_converter.sv
outputs_bcd/iter4/response0/binary_to_bcd_converter.sv
outputs_bcd/iter5/response0/binary_to_bcd_converter.sv
/content
```

Example2: traffic light

The FSM cycles through RED (32 cycles), GREEN (20 cycles), and YELLOW (7 cycles).

```
# 1) prompt
prompt = textwrap.dedent(r"""
I want to implement a traffic light FSM in Verilog-2001.

Module name must be exactly: traffic_light_fsm

Interface must match exactly:
module traffic_light_fsm(
    input  clk,
    input  reset_n,
    input  enable,
    output reg red,
    output reg yellow,
    output reg green
);

Behavior:
- Start in RED after reset_n is released (reset_n goes high).
- If enable=1, cycle:
    RED for 32 clock cycles,
    then GREEN for 20 clock cycles,
    then YELLOW for 7 clock cycles,
    then back to RED (repeat).
- If enable=0, HOLD the current state and counter (no progress).

Coding rules:
- Verilog-2001 only: use reg/wire/integer/localparam. Do NOT use logic/typedef/enum/always_ff/always_comb.
- No delays (#) or initial blocks inside the DUT.
- Use a synchronous counter and a simple state register.

Output format:
- Output ONLY the complete module code. No explanations, no markdown.
""").strip()
```

```

✓ ... Model ID: gpt-4o-mini
      Number of responses: 1
      Simulation error
      Mismatches: 30
      Samples: 705
      Cost for response 0: $0.0004411500
      Response ranks: [0.9574468085106383]
      Response lengths: [956]
      Iteration: 2
      Model type: ChatGPT
      Model ID: gpt-4o-mini
      Number of responses: 1
      Simulation error
      Mismatches: 32
      Samples: 705
      Cost for response 0: $0.0004381500
      Response ranks: [0.9546099290780142]
      Response lengths: [921]
      Iteration: 3
      Model type: ChatGPT
      Model ID: gpt-4o-mini
      Number of responses: 1
      Simulation error
      Mismatches: 32
      Samples: 705
      Cost for response 0: $0.0004273500
      Response ranks: [0.9546099290780142]
      Response lengths: [889]
      Iteration: 4
      Model type: ChatGPT
      Model ID: gpt-4o-mini
      Number of responses: 1
      Testbench ran successfully
      Mismatches: 0
      Samples: 705
      Cost for response 0: $0.0004201500
      Response ranks: [1.0]
      Response lengths: [875]

      Time to Generate: 34.39 seconds
      Best response summary:
      <__main__.LLMResponse object at 0x7ff39ffa25d0>

      Find generated modules:
      outputs_tl/iter0/response0/traffic_light_fsm.sv
      outputs_tl/iter1/response0/traffic_light_fsm.sv
      outputs_tl/iter2/response0/traffic_light_fsm.sv
      outputs_tl/iter3/response0/traffic_light_fsm.sv
      outputs_tl/iter4/response0/traffic_light_fsm.sv
      outputs_tl/iter5/response0/traffic_light_fsm.sv
      /content
      <__main__.LLMResponse at 0x7ff39ffa25d0>

```

he first few iterations resulted in compilation warnings and mismatches. After several refinement cycles, iteration 4 runs successfully.

Part I (b)

```

import os, textwrap

base = "/content/binary_to_bcd_example_partb"
os.makedirs(base, exist_ok=True)

dut = textwrap.dedent("""
module binary_to_bcd_converter(
    input  [4:0] binary_input,
    output reg [7:0] bcd_output
);
    integer i;
    reg [7:0] tmp;

    always @(*) begin
        tmp = 8'b0;
        for (i = 0; i < 5; i = i + 1) begin
            if (tmp[3:0] >= 4'd5) tmp[3:0] = tmp[3:0] + 4'd3;
            if (tmp[7:4] >= 4'd5) tmp[7:4] = tmp[7:4] + 4'd3;
            tmp = {tmp[6:0], binary_input[4 - i]};
        end
        bcd_output = tmp;
    end
endmodule
""").strip()

```

```

import os

base = "/content/binary_to_bcd_example_partb"
os.chdir(base)

dut_file = "binary_to_bcd.v"

!ls -lah
!iverilog -g2012 -o sim.out {dut_file} binary_to_bcd_tb.v
!vvp sim.out

```

```

... total 36K
drwxr-xr-x 2 root root 4.0K Feb 16 07:02 .
drwxr-xr-x 1 root root 4.0K Feb 16 06:58 ..
-rw-r--r-- 1 root root 658 Feb 16 06:59 binary_to_bcd.v
-rw-r--r-- 1 root root 1.2K Feb 16 06:59 binary_to_bcd_tb.v
-rw-r--r-- 1 root root 449 Feb 16 06:59 binary_to_bcd.v
-rw-r--r-- 1 root root 320 Feb 16 06:59 config.json
-rw-r--r-- 1 root root 3.1K Feb 16 07:02 my_design.vcd
-rwxr-xr-x 1 root root 5.0K Feb 16 07:02 sim.out
VCD info: dumpfile my_design.vcd opened for output.
Testing Binary-to-BCD Converter...
All test cases passed!
Samples: 32
Mismatches: 0 in 32 samples

```